

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE COMPUTO

Arreglos Bidimensionales

Practica 6

Alumno:

Guzmán Celaya Fernando

Grupo: 1CV1

Unidad de aprendizaje:

Fundamentos De Programación

Cadena como arreglo

Se realizó un programa donde se tiene una cadena, se obtiene la cadena, y se muestra la cadena, su longitud y cada carácter de la cadena.

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char cadena[30] = "Ejemplo de cadena";
    float calif[5];

    printf("La cadena: %s\n", cadena);
    printf("Tiene %d caracteres\n", strlen(cadena));

    int lon = strlen(cadena);
    for (int i = 0; i < lon; i++)
    {
        printf("%d\t%c\n", i, cadena[i]);
    }

    return 0;
}
```

Posteriormente se teste el código cambiando el tipo de dato de long pasándolo de int a unsigned long int,

```

// Includes
#include <stdio.h>
#include <string.h>

int main(void)
{
    // Variables
    char cadena[30] = "Ejemplo de cadena";
    float calif[5];
    unsigned long int lon = strlen(cadena);

    // Output
    printf("La cadena: %s\n", cadena);
    printf("Tiene %d caracteres\n", strlen(cadena));

    for (int i = 0; i < lon; i++)
    {
        printf("%d\t%c\n", i, cadena[i]);
    }

    return 0;
}

```

se ejecutó y se observó que funcionara igual

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc bidimensional.c -o ./compiled/bidimensional.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/bidimensional.out
La cadena: Ejemplo de cadena
Tiene 17 caracteres
0      E
1      j
2      e
3      m
4      p
5      l
6      o
7
8      d
9      e
10
11     c
12     a
13     d
14     e
15     n
16     a
```

Posteriormente

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc bidimensional.c -o ./compiled/bidimensional.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/bidimensional.out
La cadena: Ejemplo de cadena
Tiene 17 caracteres
0      E
1      j
2      e
3      m
4      p
5      l
6      o
7
8      d
9      e
10
11     c
12     a
13     d
14     e
15     n
16     a
```

Se ejecutó el programa

Finalmente se agregó un printf fuera del array de la cadena

```

// Includes
#include <stdio.h>
#include <string.h>

int main(void)
{
    // Variables
    char cadena[30] = "Ejemplo de cadena";
    float calif[5];
    unsigned long int lon = strlen(cadena);

    // Output
    printf("La cadena: %s\n", cadena);
    printf("Tiene %d caracteres\n", strlen(cadena));

    for (int i = 0; i < lon; i++)
    {
        printf("%d\t%c\n", i, cadena[i]);
    }

    printf("%d\t%c\n", 17, cadena[17]);

    return 0;
}

```

char cadena[30]

Variables

Se ejecutó y se mostró que no existe error y muestra un espacio en blanco

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc bidimensional.c -o ./compiled/bidimensional.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/bidimensional.out
La cadena: Ejemplo de cadena
Tiene 17 caracteres
0      E
1      j
2      e
3      m
4      p
5      l
6      o
7
8      d
9      e
10
11     c
12     a
13     d
14     e
15     n
16     a
17
```

Array Bidimensional

Se realizó un código donde se mostrara un array dimensional fijo, donde se usa los bucles for anidados para inicializarlo y mostrarlo en la terminal

```
// Includes
#include <stdio.h>
#include <string.h>

int main(void)
{
    // Variables
    int celda[3][4];

    // Initialize the array
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            celda[i][j] = 3;
        }
    }

    // Output
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            printf("%d\t", celda[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Posteriormente se ejecuto y mostro en la terminal

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc bidimensional.c -o ./compiled/bidimensional.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/bidimensional.out
3      3      3      3
3      3      3      3
3      3      3      3
```

Se agrego un cambio en la inicialización del array bidimensional con el fin de que mostrara la siguiente tabla en la terminal

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

```

// Includes
#include <stdio.h>
#include <string.h>

int main(void)
{
    // Variables
    int celda[3][4];

    // Initialize the array
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            celda[i][j] = 4 * i + j + 1;
        }
    }

    // Output
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            printf("%d\t", celda[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Se ejecuto y se vio que se obtuviera lo mismo que la terminal

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc bidimensional.c -o ./compiled/bidimensional.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/bidimensional.out
1      2      3      4
5      6      7      8
9      10     11     12
```

Posteriormente se agrego una dimensión mas al array pasando de ser bidimensional a tridimensional, por lo que para su inicialización también se agrego un for mas al igual que para mostrarlo en la terminal

```
// Includes
#include <stdio.h>
#include <string.h>

int main(void)
{
    // Variables
    int celda[3][3][4];

    // Initialize the array
    for (int k = 0; k < 3; k++)
    {
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 4; j++)
            {
                celda[k][i][j] = 4 * i + j + 1;
            }
        }
    }

    // Output
    for (int k = 0; k < 3; k++)
    {
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 4; j++)
            {
                printf("%d\t", celda[k][i][j]);
            }
            printf("\n");
        }
        printf("_____\n");
    }

    return 0;
}
```

Se ejecuto el array tridimensional y al ser la terminal un entorno bidimensional se mostraron tres tablas

```
● $ gcc bidimensional.c -o ./compiled/bidimensional.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/bidimensional.out
1      2      3      4
5      6      7      8
9      10     11     12

-----
1      2      3      4
5      6      7      8
9      10     11     12

-----
1      2      3      4
5      6      7      8
9      10     11     12

-----
```

Finalmente se cambió la línea que inicializaba el array con el fin de que mostrara un aumento progresivo de 1 hasta 36

```

// Includes
#include <stdio.h>
#include <string.h>

int main(void)
{
    // Variables
    int celda[3][3][4];

    // Initialize the array
    for (int k = 0; k < 3; k++)
    {
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 4; j++)
            {
                celda[k][i][j] = 12 * k + 4 * i + j + 1;
            }
        }
    }

    // Output
    for (int k = 0; k < 3; k++)
    {
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 4; j++)
            {
                printf("%d\t", celda[k][i][j]);
            }
            printf("\n");
        }
        printf("_____ \n");
    }

    return 0;
}

```

Se ejecuto y se vio el resultado

```
● $ gcc bidimensional.c -o ./compiled/bidimensional.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/bidimensional.out
1      2      3      4
5      6      7      8
9      10     11     12

-----
13     14     15     16
17     18     19     20
21     22     23     24

-----
25     26     27     28
29     30     31     32
33     34     35     36

-----
```

En el código de array unidimensional se realizaron dos funciones las cuales se encargan de buscar el valor máximo y mínimo en el array.

```
// Maximo y minimo elemento en el Array
void MaxValue(float arr[N])
{
    float fijo = arr[0];

    for (int i = 0; i < N; i++)
    {
        if (fijo < arr[i])
        {
            fijo = arr[i];
        }
    }

    printf("La calificacion maxima es: %.2f\n", fijo);
}

void MinValue(float arr[N])
{
    float fijo = arr[0];

    for (int i = 0; i < N; i++)
    {
        if (fijo > arr[i])
        {
            fijo = arr[i];
        }
    }

    printf("La calificacion minima es: %.2f\n", fijo);
}
```

```

// Main function
int main(void)
{
    // Variables
    float calif[N];

    // Function Calling
    inicializar(calif);
    mostrar(calif);
    Promedio(calif);
    desviacionEstandar(calif);
    MaxValue(calif);
    MinValue(calif);
}

```

Se ejecuto y se observaron los resultados

```

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc arrays.c -o ./compiled/arrays.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/arrays.out
Ingresa la calificacion del alumno 1: 3
Ingresa la calificacion del alumno 2: 4
Ingresa la calificacion del alumno 3: 5
Ingresa la calificacion del alumno 4: 1
Ingresa la calificacion del alumno 5: 2
Calificacion 1 : 3.00
Calificacion 2 : 4.00
Calificacion 3 : 5.00
Calificacion 4 : 1.00
Calificacion 5 : 2.00
El promedio de las calificaciones es: 3.00
La desviacion estandar es: 80.95
La calificacion maxima es: 5.00
La calificacion minima es: 1.00

```

Se realizo un código el cual escribia en un archivo ppm el tamaño de bits y mas

```

// Defines
#define WIDTH 800
#define HEIGHT 600
#define NUM_COLORS 5

// Includes
#include <stdio.h>

// Global Variables
int colot[NUM_COLORS][3] = {
    {0, 0, 0},
    {64, 64, 64},
    {128, 128, 128},
    {192, 192, 192},
    {255, 255, 255}};

int red[3] = {255, 0, 0};
int pixel[WIDTH][HEIGHT][3];

// Functions
// Pixel Printing
void PrintPixels()
{
    for (int i = 0; i < WIDTH; i++)
    {
        for (int j = 0; j < HEIGHT; j++)
        {
            for (int k = 0; k < 3; k++)
            {
                printf("%d ", pixel[i][j][k]);
            }
        }
        printf("\n");
    }
}

```

```

FILE *InitArchive()
{
    FILE *archive = fopen("out.ppm", "w");
    if (!archive)
    {
        printf("ERROR al abrir el archivo\n");
        return archive;
    }

    fprintf(archive, "P3\n");
    fprintf(archive, "%d %d \n", WIDTH, HEIGHT);
    fprintf(archive, "%d\n", 255);

    return archive;
}

// Save Image
int SaveImage()
{
    FILE *archive = InitArchive();
    if (!archive)
        return 0;

    for (int i = 0; i < HEIGHT; i++)
    {
        for (int j = 0; j < WIDTH; j++)
        {
            for (int k = 0; k < 3; k++)
            {
                fprintf(archive, "%d", pixel[i][j][k]);
            }
            fprintf(archive, "\n");
        }
    }
    fclose(archive);
    return 1;
}

```

```
// Main Function
int main()
{
    | PrintPixels();
    SaveImage();
    return 0;
}
```

Su ejecución fue la siguiente

```
escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc color.c -o ./compiled/color.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/color.out
```



```

// Draw Fil
void DrawFillRectangle(int x1, int y1, int x2, int y2, int c[3])
{
    for (int i = x1; i < x2; i++)
    {
        for (int j = y1; j < y2; j++)
        {
            pixel[i][j][0] = c[0];
            pixel[i][j][1] = c[1];
            pixel[i][j][2] = c[2];
        }
    }
}

// Main Function
int main()
{
    DrawFillRectangle(5, 5, 10, 10, red);

    // PrintPixels();
    SaveImage();
    return 0;
}

```

Su ejecución fue la siguiente

```

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ gcc color.c -o ./compiled/color.out

escom@L-1105-3 MINGW64 /e/Programming/Codes
● $ ./compiled/color.out

```

```

P3
10 10
255
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
000000000000000025500255002550025500
000000000000000025500255002550025500
000000000000000025500255002550025500
000000000000000025500255002550025500
000000000000000025500255002550025500
000000000000000025500255002550025500

```

Posteriormente se realizo usando las funciones anteriores el Mondrian

```

#include <stdio.h>

#define WIDTH      40*3
#define HEIGHT     60*3

double semi_width  = (double)WIDTH / 2.0;
double semi_height = (double)HEIGHT/ 2.0;

// escala de grises
#define NUM_COLORS  5
int gray_scale[NUM_COLORS][3] = {
    {0, 0, 0},
    {64, 64, 64},
    {128, 128, 128},
    {192, 192, 192},
    {255, 255, 255}
};

// double color_factor = (double)(NUM_COLORS-1) / (double)MAX_ITER;

// colores
int black[3]    = { 0, 0, 0 };
int red[3]     = { 255, 0, 0 };
int green[3]    = { 0, 255, 0 };
int blue[3]    = { 0, 0, 255 };
int cyan[3]    = { 0, 255, 255 };
int magenta[3] = { 255, 0, 255 };
int yellow[3]  = { 255, 255, 0 };
int white[3]   = { 255, 255, 255 };

// imagen
int pixel[WIDTH][HEIGHT][3];

void print_pixels(){

```

```

void print_pixels(){
    for (int j=0; j<HEIGHT; j++){
        for (int i=0; i<WIDTH; i++){
            printf("( ");
            for (int k=0; k<3; k++){
                printf("%d ", pixel[i][j][k]);
                printf(" ");
            }
            printf("\n");
        }
    }
}

FILE* init_archi(){
    FILE* archi = fopen("out.ppm", "w");
    if (!archi){
        printf("ERROR abriendo el archivo\n");
        return archi;
    }

    fprintf(archi, "P3\n"); // P3 significa que los colores están en ASCII
    fprintf(archi, "%d %d\n", WIDTH, HEIGHT); // pixeles ancho y alto
    fprintf(archi, "%d\n", 255); // color máximo
    return archi;
}

int save_image(){
    FILE *archi = init_archi();
    if (!archi) return 0;

    for (int j=0; j<HEIGHT; j++){
        for (int i=0; i<WIDTH; i++){
            for (int k=0; k<3; k++){
                fprintf(archi, "%d ", pixel[i][j][k]);
                //fprintf(archi, "\n");
            }
        }
        fclose(archi);
        return 1;
    }
}

```

```

void swap(int *a, int *b){
    int aux = *a;
    *a = *b;
    *b = aux;
}

void draw_fill_rect(int x1, int y1, int x2, int y2, int c[3]){
    if (x1>x2) swap(&x1, &x2);
    if (y1>y2) swap(&y1, &y2);

    if (x1<0) x1=0;
    if (y1<0) y1=0;

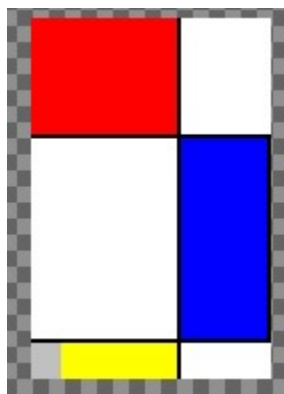
    for (int i=x1; i<=x2 && i<WIDTH; i++)
        for (int j=y1; j<=y2 && j<HEIGHT; j++){
            pixel[i][j][0] = c[0];
            pixel[i][j][1] = c[1];
            pixel[i][j][2] = c[2];
        }
}

int main(){

    draw_fill_rect(0, 0, 24*3, 19*3, red);
    draw_fill_rect(25*3, 0, 40*3, 19*3, white);
    draw_fill_rect(0, 20*3, 24*3, 53*3, white);
    draw_fill_rect(25*3, 20*3, 39*3, 53*3, blue);
    draw_fill_rect(0, 54*3, 5*3, 60*3, gray_scale[3]);
    draw_fill_rect(5*3, 54*3, 24*3, 60*3, yellow);
    draw_fill_rect(25*3, 54*3, 40*3, 60*3, white);

    //print_pixels();
    if (!save_image()) return 1;
}

```



Al igual que se realizó una variación en los colores con el fin de generar una imagen propia (en este caso un tipo Mondrian con distintos colores)

```

3 // colores
4 int vino[3] = { 133, 33, 120 };
5 int morado[3] = { 159, 146, 228 };
6 int rojito[3] = { 207, 60, 135 };
7 int verGris[3] = { 177, 190, 170 };
8 int cafe[3] = { 145, 112, 126 };
9 int moradoOsc[3] = { 47, 41, 137 };
0 int aqua[3] = { 96, 160, 165 };
1 int navy[3] = { 14, 48, 141 };
2

```

```

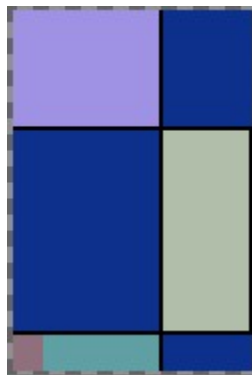
int main(){

    draw_fill_rect(0, 0, 24*3, 19*3, morado );
    draw_fill_rect(25*3, 0, 40*3, 19*3, navy);
    draw_fill_rect(0, 20*3, 24*3, 53*3, navy);
    draw_fill_rect(25*3, 20*3, 39*3, 53*3, verGris);
    draw_fill_rect(0, 54*3, 5*3, 60*3, cafe);
    draw_fill_rect(5*3, 54*3, 24*3, 60*3, aqua);
    draw_fill_rect(25*3, 54*3, 40*3, 60*3, navy);

    //print_pixels();
    if (!save_image()) return 1;

    return 0;
}

```



Finalmente se realizo un código el cual dibujaba un degradado

```
#include <stdio.h>

int main(){
    int width = 100;
    int height = 100;

    FILE* archi = fopen(fileName: "out.ppm", Mode: "w");
    if (!archi){
        printf(Format: "ERROR abriendo el archivo\n");
        return 1;
    }

    fprintf(Stream: archi, Format: "P3\n");           // P3 significa que los colores están en ASCII
    fprintf(Stream: archi, Format: "%d %d\n", width, height); // pixeles ancho y alto
    fprintf(Stream: archi, Format: "%d\n", 255);      // color máximo

    // valores RGB
    for (int j=height-1; j>=0; j--){
        for (int i=0; i<width; i++){
            int r = (int)(255.0*i/width);
            int g = (int)(255.0*j/height);
            int b = (int)(255.0*0.2);
            fprintf(Stream: archi, Format: "%d %d %d ", r, g, b);
        }
        fprintf(Stream: archi, Format: "\n");
    }

    fclose(Stream: archi);
}
```

PPM Viewer

Choose a PPM image to view

Examinar... out.ppm
 Reload image

File: out.ppm



Explicaciones:

Operaciones de arreglos:

- Suma y resta de arrays:

Se realiza sumando o restando los elementos correspondientes de dos matrices del mismo tamaño.

Ejemplo:

$$C[i][j] = A[i][j] - B[i][j].$$

- Producto de un array por un escalar:

Cada elemento de la matriz se multiplica por un número (escalar).

Ejemplo:

$$C[i][j] = k \cdot A[i][j], \text{ donde } k \text{ es el escalar.}$$

- Producto de un arreglo(Matriz) por un vector:

Cada renglón de la matriz se multiplica por los elementos correspondientes del vector, y luego se suman los resultados para formar un vector columna.

- Producto de un vector renglón de tamaño M y una matriz $M \times N$:

Cada elemento del vector renglón multiplica su columna correspondiente en la matriz, y se suman los resultados para formar un vector renglón.

- Producto de una matriz $M \times N$ por otra matriz de tamaño $N \times P$:

Cada renglón de la primera matriz se multiplica por cada columna de la segunda matriz, y los resultados se suman para formar los elementos de la matriz resultante $M \times P$

Formato de imagen ppm

PPM es un formato de imagen básico pero potente que ofrece una excelente plataforma para aprender sobre el procesamiento de imágenes y experimentar con diferentes algoritmos.

Reseña de Piet Mondrian

Piet Mondrian fue un revolucionario del arte que nos dejó un legado de belleza y simplicidad. Su obra sigue siendo relevante y inspiradora hoy en día.